

Platfomr Dokumentation

Github: <https://github.com/Mais1492/Platformer>

Version: 1.0 **Datum:** 18.05.2026 **Engine:** Godot 4.6

1. Allgemeine Informationen

- **Spielname:** Platfomr
 - **Genre:** Roguelite 3D Platformer
 - **Entwickler:** Felix Eibl, Süleyman Flachberger, Michael Mayr
 - **Ziel:** Es gilt 2 Jump & Run Levels innerhalb des Zeitlimits zu meistern. Hierzu muss jeweils ein Ziel-Ring eingesammelt werden.
-

2. Steuerung

Aktion

Tastatur

Vorwärts / Links / Rückwärts / Rechts W / A / S / D | UP / Left / Down / Right

Springen

Leertaste

Kamera drehen

Mausbewegung

3. Spielcharakter

- **Name:** Gustav (Katze) & Giacomo (Hai)
 - **Stats/Fähigkeiten:**
 - Geschwindigkeit
 - Sprunghöhe
 - Beschleunigung
 - Double-Jump: freigeschalten durch Level-Up
-

4. Features

- **2 Levels bestehend aus:**
 - Plattformen
 - Collectibles
 - Killplane

- Unterschiedlicher Hintergrundmusik & SFX
- **Plattformtypen:**
 - Normale Plattformen
 - Zerbrechliche Plattformen: verschwinden kurz nach Kontakt mit Spieler
 - Bewegende Plattformen: animierte Bewegung der Plattform
 - Eis-Plattformen: niedrige „Reibung“
 - Feder-Plattformen: Extra-Sprungkraft auf der Plattform
 - Geschwindigkeits-Plattform: Extra Geschwindigkeit auf der Plattform
 - Sprung-Plattform: gelbe Partikel schießen den Spieler automatisch nach oben
- **Collectibles mit Soundeffekten**
 - Pinke Ölfässer: erhöhen Score
 - Geschwindigkeits-Pille: erhöht Geschwindigkeit
 - Ziel-Ring: beendet das Level
- **Menüs**
 - Main Menu
 - Win Screen
- **UI**
 - Score: zählt bei jedem Durchgang weiter
 - Level: alle 3 Punkte
 - Timer: 30s
 - Double-Jump Unlock

5. Zusatz-Features

- **Kamera-System:** Ursprünglich wollten wir mit der Node „SpringArm3D“ umsetzen, dass die Kamera an den Charakter heranzoomt, wenn ein Objekt zwischen Kamera und Spieler gerät. Aus Spieldesign-Gründen haben wir uns aber gegen diesen Ansatz entschieden. Stattdessen haben wir uns eine Kamera mit einer fixen Entfernung zum Charakter entschieden, die per Mausbewegung rotiert werden kann.
- **Sichtbarkeits-Shader:** Das Anzeigen des Charakters hinter Objekten haben wir dann mit einem Outline-Shader umgesetzt. Leider wird dieser jetzt permanent angezeigt und auch Obstruktionen am Charakter selbst führen zum Shading.
- **Beleuchtung:** Wir haben versucht einen Day/Night Cycle mit dynamischem Himmel (Node: World Environment) einzubauen, haben es aber nicht geschafft

die Schatten richtig aufzulösen (wenn man das „day_night_cycle“ Script mit der Node verknüpft, werden Schatten nach oben geworfen, statt nach unten). Stattdessen haben wir uns für einen statischen Background mit der World Environment Node entschieden.

- **Physik:** Der Charakter und die Spielwelt verwenden verschiedene physikalische Faktoren, um der Bewegung „Gewicht“ zu verleihen. Folgende Werte sind parametrisiert und können angepasst werden:
 - Geschwindigkeit
 - Beschleunigung
 - Sprunghöhe
 - Reibung
 - Schwerkraft
- **Level-Up Mechanik:** Nach dem Sammeln von 3 Ölfässern erhält der Spieler einen Level-Up und schaltet die Double-Jump Fähigkeit frei. Die „on_level_up“ Methode (ui.gd) ermöglicht auch das Erhöhen der oben genannten Stats (auskommentiert, da wir keine gute Balance gefunden haben).
- **Stats Persistierung:** Die Defaultwerte der oben genannten Stats des Charakters werden in einem File "res://src/character/player_stats.tres" definiert. Wenn in runs Änderungen vorgenommen werden, so wird eine Kopie des Files mit den aktuellen Stats in "user://src/character/player_stats.tres" gespeichert und diese werden beim nächsten Run bevorzugt geladen.
- **Grid-Map Assets:** Die Plattformen wurden mit den unterschiedlichen Texturen erstellt und stehen als .tres-Files zur Verfügung. Das ermöglicht das „Zeichnen“ von Levels mit einer Grid-Map.